

SHORLYNOT Q-EDGE

DETERMINISTIC MASTER PRODUCT REQUIREMENTS DOCUMENT

Quantum-Key-State-Bound Network Enforcement and Cryptographic Agility Platform

Version: 1.0.0

Build Codename: Q-EDGE

Primary Language: Python 3.12

Edge Language: C17

Frontend: React 19 + TypeScript 5

Database: PostgreSQL 16

Cache / Event Coordination: Redis 7

Development OS: Ubuntu 24.04 LTS or Windows 11 with WSL2 Ubuntu 24.04

Edge Target: OpenWrt-compatible D-Link DSL-2750U only after exact hardware revision verification

Container Runtime: Docker Compose for development services only

Quantum SDK: Qiskit 2.x

PQC Provider: liboqs / oqs-provider

API Framework: FastAPI

ORM: SQLAlchemy 2.x async

Migration Tool: Alembic

Validation: Pydantic v2

Testing: pytest

Python Static Analysis: mypy strict + Ruff

C Static Analysis: clang-tidy + cppcheck

Frontend Testing: Vitest + Playwright

API Contract: OpenAPI 3.1 generated from FastAPI

Observability: Prometheus metrics + structured JSON logs

0. AGENT EXECUTION CONTRACT

THIS DOCUMENT IS AN EXECUTABLE ARCHITECTURAL SPECIFICATION.

THE AGENT MUST NOT:

1. Replace a required implementation with mock data.
2. Create hardcoded dashboard metrics.
3. Generate fake quantum job IDs.
4. Generate fake QKD hardware telemetry.
5. Label pseudo-random values as quantum measurements.
6. Label random bytes as ML-KEM.
7. Change a UI state without executing the corresponding backend operation.
8. Simulate firewall execution and report success.

9. Silently fall back from real QKD hardware to a simulator.
10. silently fall back from real QPU execution to local execution.
11. create placeholder functions containing `pass`.
12. create TODO-only implementations.
13. swallow exceptions with `except Exception: pass`.
14. return HTTP 200 for failed operations.
15. store raw cryptographic secrets in PostgreSQL.
16. log private keys, shared secrets, session encryption keys, or QKD key material.
17. invent an OpenWrt target for an unsupported router revision.
18. flash router firmware automatically.
19. claim physical QKD unless a physical QKD backend reports authenticated hardware provenance.
20. modify an existing file without reading the complete file first.

Every security claim visible in the UI must map to an executed backend operation and an immutable audit event.

1. PRODUCT DEFINITION

ShorlyNot Q-Edge is a quantum-security-state-aware edge enforcement platform.

The system consumes authenticated security state from:

1. a BB84 protocol engine;
2. a physical QKD hardware adapter when hardware is available;
3. a real quantum processor adapter for quantum measurement experiments;
4. a real ML-KEM implementation;
5. cryptographic asset scanners.

The system converts this state into:

1. key acceptance or rejection;
2. cryptographic mode selection;
3. temporary network authorization leases;
4. firewall policy installation;
5. firewall policy revocation;
6. session rekeying;
7. QKD-to-PQC migration;
8. source isolation;
9. lockdown.

The primary mechanism is named:

QUANTUM KEY-BOUND NETWORK LEASE

Abbreviation:

QKBNL

A QKBNL is a temporary network forwarding authorization associated with:

- session identity;
- source identity;
- destination identity;
- source IP;
- destination IP;
- protocol;
- destination port;
- cryptographic mode;
- key lifecycle state;
- channel trust state;
- expiry timestamp.

Network access is not permanently granted.

A valid QKBNL must exist before protected traffic is forwarded.

2. PRIMARY SYSTEM INVARIANTS

INVARIANT QEDGE-001

A rejected QKD key must never create a network lease.

INVARIANT QEDGE-002

An expired key must never maintain an active network lease.

INVARIANT QEDGE-003

A revoked session must have all associated network leases removed.

INVARIANT QEDGE-004

Invalid telemetry must never transition a channel to TRUSTED.

INVARIANT QEDGE-005

A QKD key with lifecycle state REJECTED must never enter the hybrid key combiner.

INVARIANT QEDGE-006

A QKD key with lifecycle state ZEROIZED must never be returned by any API.

INVARIANT QEDGE-007

Protected traffic must not resume until peer key confirmation succeeds.

INVARIANT QEDGE-008

A firewall operation is successful only when the target edge agent executes the rule and returns a signed execution receipt.

INVARIANT QEDGE-009

The dashboard must display backend state only.

INVARIANT QEDGE-010

Physical QKD status is AVAILABLE only after a configured physical QKD adapter completes authentication and hardware identity verification.

INVARIANT QEDGE-011

Real QPU status is AVAILABLE only after the configured provider returns a valid backend identity.

INVARIANT QEDGE-012

No secret key bytes may appear in application logs.

3. PHYSICAL DEPLOYMENT

Node A:

Your primary laptop.

Role:

CONTROL NODE

Runs:

- PostgreSQL
- Redis
- FastAPI control plane

- trust engine
- policy engine
- key lifecycle service
- ML-KEM service
- QKD adapter manager
- quantum backend manager
- audit service
- Prometheus exporter
- dashboard

Node B:

Second laptop.

Role:

ALICE NODE

Runs:

- Alice BB84 client
- secure workload client
- QTP telemetry producer
- node identity agent
- ML-KEM client
- encrypted session client

Node C:

Friend's laptop.

Role:

BOB NODE

Runs:

- Bob BB84 client
- financial workload services
- QTP telemetry producer
- node identity agent
- ML-KEM server
- encrypted session server

Node D:

D-Link DSL-2750U.

Role:

EDGE ENFORCEMENT NODE

Runs only after compatibility verification:

- OpenWrt base
- qtp-edge
- qlease
- qpolicy
- qreceipt
- firewall integration
- procd services

All protected Alice-to-Bob traffic must physically traverse Node D.

The application must verify this by recording source route information and by proving traffic interruption when the D-Link forwarding policy is removed.

4. NETWORK ADDRESS PLAN

Development physical network:

CONTROL NODE: 10.77.0.10/24

ALICE NODE: 10.77.0.20/24

BOB NODE: 10.77.0.30/24

EDGE MANAGEMENT: 10.77.0.1/24

Redis: 10.77.0.10:6379

PostgreSQL: 10.77.0.10:5432

Control API: 10.77.0.10:8443

QTP ingest: 10.77.0.10:9443

Bob payment API: 10.77.0.30:8101

Bob identity API: 10.77.0.30:8102

Bob interbank service: 10.77.0.30:8103

Bob archive service: 10.77.0.30:8104

Edge agent: 10.77.0.1:7443

No component may assume `localhost` for inter-node communication.

All addresses must be configurable through environment variables.

5. ENVIRONMENT VARIABLES

Create `.env.example`.

Required variables:

APP_ENV: Literal["development", "test", "production"]

APP_NAME: string Default: "shorlynot-qedge"

CONTROL_API_HOST: IPv4 or hostname CONTROL_API_PORT: integer 1-65535

DATABASE_URL: PostgreSQL async DSN

REDIS_URL: Redis DSN

CONTROL_NODE_ID: UUID

CONTROL_TLS_CERT_PATH: absolute filesystem path CONTROL_TLS_KEY_PATH: absolute filesystem path

CONTROL_CA_CERT_PATH: absolute filesystem path

QTP_HOST: IPv4 or hostname QTP_PORT: integer 1-65535

EDGE_AGENT_URL: HTTPS URL

EDGE_NODE_ID: UUID

EDGE_RECEIPT_PUBLIC_KEY_PATH: absolute filesystem path

OQS_PROVIDER_PATH: absolute filesystem path

ML_KEM_ALGORITHM: exact value "ML-KEM-768"

ML_DSA_ALGORITHM: exact configured supported ML-DSA identifier

QKD_BACKEND: Literal["unavailable", "bb84_protocol", "physical_qkd"]

QUANTUM_BACKEND: Literal["unavailable", "ibm_quantum"]

IBM_QUANTUM_TOKEN: optional secret string

IBM_QUANTUM_INSTANCE: optional string

IBM_QUANTUM_BACKEND: optional string

QKD_VENDOR: optional string

QKD_API_BASE_URL: optional HTTPS URL

QKD_API_CLIENT_CERT: optional path

QKD_API_CLIENT_KEY: optional path

QKD_API_CA_CERT: optional path

QKD_ADAPTER_TYPE: Literal["none", "etsi_qkd_014", "vendor_custom"]

TELEMETRY_MAX_AGE_SECONDS: integer Default: 10 Minimum: 1 Maximum: 300

TELEMETRY_REPLAY_TTL_SECONDS: integer Default: 300

LEASE_DEFAULT_TTL_SECONDS: integer Default: 120 Minimum: 5 Maximum: 3600

TRUST_WINDOW_SIZE: integer Default: 20 Minimum: 5 Maximum: 10000

EWMA_ALPHA: float Default: 0.3 Exclusive minimum: 0 Maximum: 1

CUSUM_DRIFT: float Default: 0.005 Minimum: 0

CUSUM_THRESHOLD: float Default: 0.03 Exclusive minimum: 0

STATIC_QBER_REJECT_THRESHOLD: float Default: 0.11 Minimum: 0 Maximum: 1

LOG_LEVEL: Literal["DEBUG", "INFO", "WARNING", "ERROR", "CRITICAL"]

Secrets must never be committed.

`.env` must be in `.gitignore`.

Application startup must fail if a required variable is missing.

6. REPOSITORY STRUCTURE

Create exactly:

```
shorlynot-qedge/ ├── README.md ├── LICENSE ├── SECURITY.md ├── CONTRIBUTING.md
│ ├── Makefile ├── pyproject.toml ├── docker-compose.yml ├── .env.example ├── .gitignore
│ ├── alembic.ini ├── control_plane/ │ │ ├── init.py │ │ ├── main.py │ │ ├── config.py │ │ └──
lifespan.py │ │ ├── dependencies.py │ │ ├── exceptions.py │ │ ├── logging_config.py │ │ └──
database/ │ │ │ ├── base.py │ │ │ ├── session.py │ │ │ └── models/ │ │ │ │ │ ├── node.py │ │ │ │ │ └──
channel.py │ │ │ ├── qkd_session.py │ │ │ ├── key_record.py │ │ │ ├── telemetry.py │ │ │ └──
trust_decision.py │ │ │ ├── crypto_session.py │ │ │ ├── network_lease.py │ │ │ └──
enforcement_receipt.py │ │ │ ├── crypto_asset.py │ │ │ ├── migration_plan.py │ │ │ └──
audit_event.py │ │ ├── schemas/ │ │ │ ├── common.py │ │ │ ├── node.py │ │ │ ├── channel.py │ │ │ └──
│ │ │ ├── qkd.py │ │ │ ├── telemetry.py │ │ │ ├── trust.py │ │ │ ├── crypto.py │ │ │ ├── lease.py │ │ │ └──
│ │ │ ├── enforcement.py │ │ │ ├── assets.py │ │ │ ├── migration.py │ │ │ └── repositories/ │ │ │ │ │ ├──
node_repository.py │ │ │ │ │ ├── channel_repository.py │ │ │ │ │ ├── qkd_repository.py │ │ │ │ │ └──
key_repository.py │ │ │ │ │ ├── telemetry_repository.py │ │ │ │ │ ├── trust_repository.py │ │ │ │ │ └──
crypto_repository.py │ │ │ │ │ ├── lease_repository.py │ │ │ │ │ ├── enforcement_repository.py │ │ │ │ │ └──
asset_repository.py │ │ │ │ │ ├── migration_repository.py │ │ │ │ │ ├── audit_repository.py │ │ │ │ │ └── services/
│ │ │ │ │ │ ├── node_service.py │ │ │ │ │ │ ├── bb84_service.py │ │ │ │ │ │ ├── reconciliation_service.py │ │ │ │ │ │ └──
privacy_service.py │ │ │ │ │ │ ├── qkd_adapter_service.py │ │ │ │ │ │ ├── quantum_backend_service.py │ │ │ │ │ │ └──
│ │ │ │ │ │ ├── telemetry_service.py │ │ │ │ │ │ ├── trust_service.py │ │ │ │ │ │ ├── mlkem_service.py │ │ │ │ │ │ └──
mldsa_service.py │ │ │ │ │ │ ├── hybrid_kdf_service.py │ │ │ │ │ │ ├── key_lifecycle_service.py │ │ │ │ │ │ └──
crypto_session_service.py │ │ │ │ │ │ ├── policy_service.py │ │ │ │ │ │ ├── lease_service.py │ │ │ │ │ │ └──
enforcement_service.py │ │ │ │ │ │ ├── asset_discovery_service.py │ │ │ │ │ │ ├── risk_service.py │ │ │ │ │ │ └──
migration_service.py │ │ │ │ │ │ ├── audit_service.py │ │ │ │ │ │ ├── routers/ │ │ │ │ │ │ │ ├── health_router.py │ │ │ │ │ │ │ └──
│ │ │ │ │ │ │ ├── node_router.py │ │ │ │ │ │ │ ├── channel_router.py │ │ │ │ │ │ │ ├── qkd_router.py │ │ │ │ │ │ │ └──
telemetry_router.py │ │ │ │ │ │ │ ├── trust_router.py │ │ │ │ │ │ │ ├── crypto_router.py │ │ │ │ │ │ │ ├── lease_router.py │ │ │ │ │ │ │ └──
│ │ │ │ │ │ │ ├── enforcement_router.py │ │ │ │ │ │ │ ├── asset_router.py │ │ │ │ │ │ │ ├── migration_router.py │ │ │ │ │ │ │ └──
audit_router.py │ │ │ │ │ │ ├── adapters/ │ │ │ │ │ │ │ ├── qkd/ │ │ │ │ │ │ │ │ ├── base.py │ │ │ │ │ │ │ │ ├── unavailable.py │ │ │ │ │ │ │ │ └──
│ │ │ │ │ │ │ │ ├── bb84_protocol.py │ │ │ │ │ │ │ │ ├── etsi_qkd_014.py │ │ │ │ │ │ │ │ ├── quantum/ │ │ │ │ │ │ │ │ │ ├── base.py │ │ │ │ │ │ │ │ │ └──
│ │ │ │ │ │ │ │ │ ├── unavailable.py │ │ │ │ │ │ │ │ │ ├── ibm_quantum.py │ │ │ │ │ │ │ │ │ ├── edge/ │ │ │ │ │ │ │ │ │ ├── base.py │ │ │ │ │ │ │ │ │ └──
│ │ │ │ │ │ │ │ │ ├── openwrt_edge.py │ │ │ │ │ │ │ │ │ ├── trust/ │ │ │ │ │ │ │ │ │ ├── ewma.py │ │ │ │ │ │ │ │ │ ├── cusum.py │ │ │ │ │ │ │ │ │ └──
rolling_variance.py │ │ │ │ │ │ │ │ │ ├── change_point.py │ │ │ │ │ │ │ │ │ ├── evidence_engine.py │ │ │ │ │ │ │ │ │ ├── crypto/ │ │ │ │ │ │ │ │ │ └──
│ │ │ │ │ │ │ │ │ │ ├── secure_memory.py │ │ │ │ │ │ │ │ │ │ ├── key_combiner.py │ │ │ │ │ │ │ │ │ │ ├── aesgcm.py │ │ │ │ │ │ │ │ │ │ ├── discovery/ │ │ │ │ │ │ │ │ │ │ └──
│ │ │ │ │ │ │ │ │ │ ├── certificate_scanner.py │ │ │ │ │ │ │ │ │ │ ├── source_scanner.py │ │ │ │ │ │ │ │ │ │ ├── tls_scanner.py │ │ │ │ │ │ │ │ │ │ └──
config_scanner.py │ │ │ │ │ │ │ │ │ │ ├── middleware/ │ │ │ │ │ │ │ │ │ │ ├── request_id.py │ │ │ │ │ │ │ │ │ │ ├── security_headers.py │ │ │ │ │ │ │ │ │ │ └──
rate_limit.py │ │ │ │ │ │ │ │ │ │ ├── nodes/ │ │ │ │ │ │ │ │ │ │ ├── common/ │ │ │ │ │ │ │ │ │ │ ├── identity.py │ │ │ │ │ │ │ │ │ │ ├── qtp_client.py │ │ │ │ │ │ │ │ │ │ └──
secure_channel.py │ │ │ │ │ │ │ │ │ │ ├── alice/ │ │ │ │ │ │ │ │ │ │ ├── main.py │ │ │ │ │ │ │ │ │ │ ├── bb84_alice.py │ │ │ │ │ │ │ │ │ │ └──
workload_client.py │ │ │ │ │ │ │ │ │ │ ├── bob/ │ │ │ │ │ │ │ │ │ │ ├── main.py │ │ │ │ │ │ │ │ │ │ ├── bb84_bob.py │ │ │ │ │ │ │ │ │ │ ├── workload_server.py
│ │ │ │ │ │ │ │ │ │ ├── edgeos/ │ │ │ │ │ │ │ │ │ │ ├── README.md │ │ │ │ │ │ │ │ │ │ ├── compatibility/ │ │ │ │ │ │ │ │ │ │ ├── supported_devices.json │ │ │ │ │ │ │ │ │ │ └──
package/ │ │ │ │ │ │ │ │ │ │ ├── qedge-agent/ │ │ │ │ │ │ │ │ │ │ ├── Makefile │ │ │ │ │ │ │ │ │ │ ├── src/ │ │ │ │ │ │ │ │ │ │ ├── main.c │ │ │ │ │ │ │ │ │ │ └──
│ │ │ │ │ │ │ │ │ │ │ ├── config.c │ │ │ │ │ │ │ │ │ │ │ ├── config.h │ │ │ │ │ │ │ │ │ │ │ ├── protocol.c │ │ │ │ │ │ │ │ │ │ │ ├── protocol.h │ │ │ │ │ │ │ │ │ │ │ └──
│ │ │ │ │ │ │ │ │ │ │ ├── lease.c │ │ │ │ │ │ │ │ │ │ │ ├── lease.h │ │ │ │ │ │ │ │ │ │ │ ├── firewall.c │ │ │ │ │ │ │ │ │ │ │ ├── firewall.h │ │ │ │ │ │ │ │ │ │ │ └──
```

```

| | ├── receipt.c | | | | └── receipt.h | | | | └── files/ | | | | ├── qedge-agent.init | | | |
└── qedge-agent.config | └── scripts/ | ├── verify_target.sh | ├── build_image.sh | └──
verify_image.sh ├── workloads/ | ├── payment/ | | | └── app.py | ├── identity/ | | | └──
app.py | ├── interbank/ | | | └── app.py | └── archive/ | └── app.py ├── experiments/ |
└── runners/ | | | ├── qber_baseline.py | | | ├── temporal_detection.py | | | └──
telemetry_attacks.py | | | ├── lease_revocation.py | | | ├── crypto_migration.py | | | └──
workload_benchmark.py | ├── plots/ | | | ├── qber_plot.py | | | ├── detection_plot.py | | | └──
lease_plot.py | | | └── migration_plot.py | └── results/ ├── dashboard/ | ├── package.json |
└── tsconfig.json | ├── vite.config.ts | └── src/ | ├── main.tsx | ├── App.tsx | ├── api/
| | | └── client.ts | ├── types/ | | | └── api.ts | ├── pages/ | | | ├── OverviewPage.tsx | |
└── QuantumPage.tsx | | | ├── ChannelsPage.tsx | | | ├── LeasesPage.tsx | | | └──
CryptoPage.tsx | | | ├── EdgePage.tsx | | | ├── AssetsPage.tsx | | | └── AuditPage.tsx | └──
components/ ├── formal/ | | | ├── QEdge.tla | └── QEdge.cfg └── tests/ ├── unit/ ├──
integration/ ├── security/ ├── edge/ └── e2e/

```

No additional top-level application directory may be created.

7. DATABASE ENUMS

NodeType:

CONTROL ALICE BOB EDGE QKD_DEVICE

NodeStatus:

PENDING ACTIVE UNTRUSTED OFFLINE REVOKED

ChannelState:

UNKNOWN TRUSTED NOISY DEGRADING SUSPICIOUS COMPROMISED

KeyState:

RAW SIFTED ESTIMATING RECONCILING AMPLIFYING CONFIRMING ACCEPTED REJECTED EXPIRED REVOKED
ZEROIZED

CryptoMode:

QKD_ONLY ML_KEM_ONLY QKD_ML_KEM_HYBRID LOCKED

LeaseState:

PENDING ACTIVE EXPIRED REVOKED FAILED

EnforcementAction:

LEASE_INSTALL LEASE_REVOKE SESSION_ISOLATE CHANNEL_ISOLATE LOCKDOWN

EnforcementStatus:

REQUESTED VERIFIED EXECUTING SUCCEEDED FAILED

AuditSeverity:

INFO WARNING HIGH CRITICAL

8. DATABASE MODELS

All primary keys use PostgreSQL UUID.

All timestamps use `TIMESTAMP WITH TIME ZONE`.

All timestamps are UTC.

nodes

id: UUID primary key

name: VARCHAR(100) not null unique

node_type: NodeType not null

status: NodeStatus not null default PENDING

ipv4_address: INET not null

identity_public_key: BYTEA not null

certificate_fingerprint: VARCHAR(128) not null unique

created_at: timestamptz not null

updated_at: timestamptz not null

revoked_at: timestamptz nullable

Delete rule:

RESTRICT if referenced.

Nodes are revoked, not deleted.

channels

id: UUID primary key

name: VARCHAR(100) not null unique

alice_node_id: UUID foreign key nodes.id ON DELETE RESTRICT

bob_node_id: UUID foreign key nodes.id ON DELETE RESTRICT

backend_type: VARCHAR(50) not null

state: ChannelState not null default UNKNOWN

current_qber: NUMERIC(8,7) nullable

current_secret_key_rate: BIGINT nullable

last_telemetry_at: timestamptz nullable

created_at: timestamptz not null

Constraint:

alice_node_id != bob_node_id

QBER constraint:

current_qber >= 0 AND current_qber <= 1

qkd_sessions

id: UUID primary key

channel_id: UUID FK channels.id ON DELETE RESTRICT

external_session_id: VARCHAR(255) nullable

backend_type: VARCHAR(50) not null

backend_provenance: JSONB not null default '{}'

started_at: timestamptz not null

completed_at: timestamptz nullable

final_qber: NUMERIC(8,7) nullable

accepted: BOOLEAN nullable

failure_code: VARCHAR(100) nullable

key_records

id: UUID primary key

qkd_session_id: UUID FK qkd_sessions.id ON DELETE RESTRICT nullable

key_fingerprint: VARCHAR(64) not null unique

state: KeyState not null

key_length_bits: INTEGER not null

qber: NUMERIC(8,7) nullable

reconciliation_leakage_bits: INTEGER not null default 0

privacy_output_bits: INTEGER nullable

created_at: timestamptz not null

expires_at: timestamptz nullable

revoked_at: timestamptz nullable

zeroized_at: timestamptz nullable

IMPORTANT:

No raw key bytes column exists.

Key bytes remain only in the secure key store process memory.

telemetry_events

id: UUID primary key

node_id: UUID FK nodes.id ON DELETE RESTRICT

channel_id: UUID FK channels.id ON DELETE RESTRICT

session_id: UUID nullable

sequence_number: BIGINT not null

event_timestamp: timestamptz not null

received_at: timestamptz not null

nonce_hash: VARCHAR(64) not null

qber: NUMERIC(8,7) not null

qber_x: NUMERIC(8,7) nullable

qber_z: NUMERIC(8,7) nullable

sifted_key_rate: BIGINT nullable

secret_key_rate: BIGINT nullable

key_pool_bits: BIGINT nullable

signature_valid: BOOLEAN not null

raw_payload_hash: VARCHAR(64) not null

Unique constraint:

(node_id, sequence_number)

Unique constraint:

(node_id, nonce_hash)

trust_decisions

id: UUID primary key

channel_id: UUID FK channels.id ON DELETE RESTRICT

telemetry_event_id: UUID FK telemetry_events.id ON DELETE RESTRICT

state: ChannelState not null

confidence: NUMERIC(5,4) not null

ewma_value: NUMERIC(8,7) nullable

cusum_positive: NUMERIC(12,8) nullable

rolling_variance: NUMERIC(12,10) nullable

evidence: JSONB not null

created_at: timestamptz not null

Confidence constraint:

confidence >= 0 AND confidence <= 1

crypto_sessions

id: UUID primary key

alice_node_id: UUID FK nodes.id ON DELETE RESTRICT

bob_node_id: UUID FK nodes.id ON DELETE RESTRICT

channel_id: UUID FK channels.id ON DELETE RESTRICT nullable

key_record_id: UUID FK key_records.id ON DELETE RESTRICT nullable

mode: CryptoMode not null

session_key_fingerprint: VARCHAR(64) not null

started_at: timestamptz not null

rekeyed_at: timestamptz nullable

terminated_at: timestamptz nullable

network_leases

id: UUID primary key

crypto_session_id: UUID FK crypto_sessions.id ON DELETE CASCADE

key_record_id: UUID FK key_records.id ON DELETE RESTRICT nullable

source_ip: INET not null

destination_ip: INET not null

protocol: VARCHAR(10) not null

destination_port: INTEGER not null

state: LeaseState not null

issued_at: timestamptz not null

expires_at: timestamptz not null

revoked_at: timestamptz nullable

edge_rule_id: VARCHAR(255) nullable

Constraint:

destination_port >= 1 AND destination_port <= 65535

Constraint:

protocol IN ('tcp','udp','icmp')

Constraint:

expires_at > issued_at

enforcement_receipts

id: UUID primary key

lease_id: UUID FK network_leases.id ON DELETE CASCADE nullable

edge_node_id: UUID FK nodes.id ON DELETE RESTRICT

action: EnforcementAction not null

status: EnforcementStatus not null

request_id: UUID not null unique

edge_rule_id: VARCHAR(255) nullable

executed_at: timestamptz nullable

receipt_payload: JSONB not null

receipt_signature: BYTEA not null

verified: BOOLEAN not null default false

created_at: timestamptz not null

crypto_assets

id: UUID primary key

asset_name: VARCHAR(255) not null

asset_type: VARCHAR(100) not null

location: TEXT not null

algorithm: VARCHAR(100) not null

key_size_bits: INTEGER nullable

certificate_expiry: timestamptz nullable

data_retention_years: INTEGER nullable

external_exposure: BOOLEAN not null

criticality: INTEGER not null

quantum_vulnerable: BOOLEAN not null

risk_score: NUMERIC(6,3) nullable

created_at: timestamptz not null

Criticality:

integer 1 through 5.

migration_plans

id: UUID primary key

asset_id: UUID FK crypto_assets.id ON DELETE CASCADE

current_algorithm: VARCHAR(100) not null

recommended_algorithm: VARCHAR(100) not null

migration_phase: INTEGER not null

priority: VARCHAR(20) not null

measured_latency_impact_ms: NUMERIC(12,4) nullable

compatibility_notes: TEXT not null

rollback_required: BOOLEAN not null

created_at: timestamptz not null

audit_events

id: UUID primary key

event_type: VARCHAR(100) not null

severity: AuditSeverity not null

actor_node_id: UUID FK nodes.id ON DELETE SET NULL nullable

resource_type: VARCHAR(100) not null

resource_id: UUID nullable

request_id: UUID nullable

payload: JSONB not null

previous_event_hash: VARCHAR(64) nullable

event_hash: VARCHAR(64) not null unique

created_at: timestamptz not null

Audit records are append-only.

Application code must not expose UPDATE or DELETE operations for audit_events.

9. API ERROR SCHEMA

Every failed API response must use:

```
{ "error": { "code": "STRING_ERROR_CODE", "message": "Human readable message", "request_id": "uuid",  
"details": {} } }
```

Never return Python stack traces.

HTTP mappings:

400: INVALID_REQUEST

401: AUTHENTICATION_REQUIRED INVALID_NODE_SIGNATURE

403: NODE_REVOKED POLICY_DENIED KEY_NOT_ACCEPTED

404: NODE_NOT_FOUND CHANNEL_NOT_FOUND SESSION_NOT_FOUND KEY_NOT_FOUND
LEASE_NOT_FOUND

409: DUPLICATE_SEQUENCE NONCE_REPLAY INVALID_STATE_TRANSITION LEASE_ALREADY_ACTIVE
SESSION_ALREADY_TERMINATED

422: SCHEMA_VALIDATION_FAILED

429: RATE_LIMIT_EXCEEDED

502: EDGE_AGENT_FAILURE QKD_BACKEND_FAILURE QUANTUM_PROVIDER_FAILURE

503: EDGE_AGENT_UNAVAILABLE QKD_BACKEND_UNAVAILABLE QUANTUM_BACKEND_UNAVAILABLE

500: INTERNAL_ERROR

10. API ENDPOINTS

GET /health

Response 200:

```
{ "status": "healthy", "database": true, "redis": true, "timestamp": "ISO-8601" }
```

Return 503 if PostgreSQL or Redis fails.

GET /api/v1/system/capabilities

Response:

```
{ "physical_qkd": { "available": false, "adapter": "none" }, "quantum_backend": { "available": true, "provider": "ibm_quantum", "backend": "actual-backend-name" }, "pqc": { "ml_kem_768": true, "ml_dsa": true }, "edge": { "available": true, "node_id": "uuid" } }
```

Values must come from runtime capability checks.

POST /api/v1/nodes

Input:

```
{ "name": "alice-01", "node_type": "ALICE", "ipv4_address": "10.77.0.20", "identity_public_key": "base64", "certificate_fingerprint": "hex" }
```

Validation:

name 1-100 characters.

Valid IPv4 required.

Public key must decode successfully.

Fingerprint must be valid hexadecimal.

Duplicate name returns 409.

Duplicate fingerprint returns 409.

GET /api/v1/nodes

Return paginated node list.

Query:

limit integer 1-100 default 50.

offset integer ≥ 0 default 0.

POST /api/v1/channels

Input:

```
{ "name": "qch-01", "alice_node_id": "uuid", "bob_node_id": "uuid", "backend_type": "bb84_protocol" }
```

Both nodes must exist.

Alice and Bob IDs must differ.

POST /api/v1/qkd/sessions

Input:

```
{ "channel_id": "uuid" }
```

The service must select the configured QKD adapter.

If physical QKD is configured but unavailable:

return 503 QKD_BACKEND_UNAVAILABLE.

Never silently switch adapters.

GET /api/v1/qkd/sessions/{session_id}

Return session metadata.

Never return raw key bytes.

POST /api/v1/telemetry

Input QTP packet.

Required fields:

protocol: exact "QTP"

version: exact integer 1

node_id: UUID

channel_id: UUID

session_id: UUID nullable

sequence: unsigned integer

timestamp: ISO-8601 UTC

qber: float 0-1

qber_x: nullable float 0-1

qber_z: nullable float 0-1

sifted_key_rate: nullable non-negative integer

secret_key_rate: nullable non-negative integer

key_pool_bits: nullable non-negative integer

nonce: base64 16-64 bytes

signature: base64

Processing order must be exactly:

1. validate JSON schema;
2. resolve node;
3. verify node status ACTIVE;
4. verify signature;
5. validate timestamp age;
6. hash nonce;
7. check nonce replay cache;
8. validate sequence number;
9. persist telemetry;
10. calculate trust;
11. persist trust decision;
12. execute policy evaluation;
13. append audit event;
14. return result.

Failure before step 9 must not persist telemetry as trusted telemetry.

Invalid signed packets may be persisted only in audit_events.

GET /api/v1/channels/{channel_id}/trust

Response:

```
{ "channel_id": "uuid", "state": "SUSPICIOUS", "confidence": 0.8700, "metrics": { "current_qber": 0.084,
"ewma": 0.067, "cusum_positive": 0.034, "rolling_variance": 0.00081 }, "evidence": [ { "code":
"CUSUM_THRESHOLD_EXCEEDED", "value": 0.034, "threshold": 0.03 } ] }
```

Evidence must use deterministic evidence codes.

POST /api/v1/crypto/sessions

Input:

```
{ "alice_node_id": "uuid", "bob_node_id": "uuid", "channel_id": "uuid" }
```

Policy engine selects mode.

Client cannot request a weaker mode.

POST /api/v1/crypto/sessions/{id}/rekey

The service must:

1. lock crypto session row;
2. verify session active;
3. determine current trust state;
4. derive new key source;
5. complete peer key confirmation;
6. atomically update session fingerprint;
7. revoke old leases;
8. issue replacement leases;
9. receive edge execution receipts;
10. resume application traffic.

Failure before step 5:

keep existing valid session if policy permits.

Failure after key rejection:

do not restore rejected QKD key.

POST /api/v1/leases

Input:

```
{ "crypto_session_id": "uuid", "source_ip": "10.77.0.20", "destination_ip": "10.77.0.30", "protocol": "tcp",  
  "destination_port": 8101, "ttl_seconds": 120 }
```

The service must reject the request if:

session terminated;

mode LOCKED;

associated QKD key REJECTED;

associated QKD key EXPIRED;

channel COMPROMISED and mode QKD_ONLY;

edge unavailable.

The database transaction must create a PENDING lease.

The lease becomes ACTIVE only after a verified edge receipt with status SUCCEEDED.

DELETE /api/v1/leases/{lease_id}

This endpoint performs revocation.

It must not delete the row.

State sequence:

ACTIVE -> REVOKED

The edge rule must be removed before the API reports successful revocation.

POST /api/v1/assets/scan

Input:

```
{ "scan_type": "source|certificate|tls|config", "target": "string" }
```

The backend validates target access.

The scanner may only scan targets explicitly submitted.

TLS scanner must not perform broad network discovery.

GET /api/v1/assets

Paginated asset results.

POST /api/v1/migration/generate

Generate migration recommendations from stored assets and benchmark data.

No LLM is used.

GET /api/v1/audit

Read-only paginated endpoint.

No POST.

No PUT.

No PATCH.

No DELETE.

11. BB84 PROTOCOL REQUIREMENTS

The BB84 protocol module must contain separate Alice and Bob logic.

Alice produces:

bit array.

basis array.

State mapping:

bit 0 + Z basis -> $|0\rangle$

bit 1 + Z basis -> $|1\rangle$

bit 0 + X basis -> $|+\rangle$

bit 1 + X basis -> | ->

Bob independently selects bases.

After measurement:

Alice and Bob exchange basis identifiers through the authenticated classical application channel.

Sifting retains indices where bases match.

Parameter estimation selects a configured sample from the sifted key.

QBER:

number of differing disclosed sample bits divided by number of disclosed sample bits.

If disclosed sample size is zero:

session fails with QBER_SAMPLE_EMPTY.

The disclosed sample bits must not remain in the candidate secret key.

The system must execute reconciliation.

The system must execute privacy amplification.

The system must execute key confirmation.

Only after key confirmation may state become ACCEPTED.

The BB84 protocol adapter must identify itself as:

BB84_PROTOCOL

It must not identify itself as PHYSICAL_QKD.

12. PHYSICAL QKD ADAPTER CONTRACT

Base adapter methods:

connect() -> CapabilityStatus

authenticate() -> AuthenticationResult

get_device_identity() -> DeviceIdentity

create_key_session(channel_id: UUID) -> ExternalSession

get_session_status(external_session_id: str) -> SessionStatus

get_qber(external_session_id: str) -> Decimal

get_key_metadata(external_session_id: str) -> KeyMetadata

request_key(external_session_id: str) -> SecureKeyHandle

close_session(external_session_id: str) -> None

Physical adapter requirements:

Mutual TLS.

Certificate validation enabled.

No `verify=False`.

API timeout 5 seconds.

Maximum 3 retries.

Retry delays:

250 ms

500 ms

1000 ms

Do not retry authentication failure.

Do not retry malformed response.

The physical QKD adapter is AVAILABLE only if:

TLS succeeds;

authentication succeeds;

device identity is returned;

adapter protocol version is supported.

The adapter must store vendor metadata in `qkd_sessions.backend_provenance`.

The key itself must remain behind `SecureKeyHandle`.

If no real device is configured:

use `unavailable.py`.

The unavailable adapter always raises `QKDBackendUnavailable`.

It must never generate a key.

13. IBM QUANTUM BACKEND CONTRACT

The IBM adapter exists for real quantum processor execution provenance.

It is not the physical QKD adapter.

Methods:

`connect()`

`list_available_backends()`

`verify_backend(name)`

`submit(circuit)`

`get_job(job_id)`

`get_result(job_id)`

`get_provenance(job_id)`

Provenance schema:

`provider: string`

`backend_name: string`

`job_id: string`

submitted_at: ISO timestamp

completed_at: ISO timestamp nullable

circuit_sha256: 64-char hex

result_sha256: 64-char hex nullable

shots: positive integer

status: string

The adapter must reject simulator backends when real-hardware mode is enabled.

The UI must display:

REAL QPU

only when adapter backend metadata confirms a non-simulator backend.

14. RECONCILIATION REQUIREMENTS

Implement a Cascade-style reconciliation service.

Input:

Alice sifted key secure handle.

Bob sifted key secure handle.

Maximum passes:

4.

Pass block sizes are configuration driven.

Each pass:

1. derive shared permutation seed;
2. permute indices;
3. partition into blocks;
4. compare parity;
5. binary search blocks with parity mismatch;

6. correct identified bit;
7. record disclosed parity information.

Output:

corrected key handles.

leakage_bits.

corrected_bits.

passes_completed.

duration_ms.

If Alice and Bob keys remain unequal after maximum passes:

session becomes REJECTED.

Failure code:

RECONCILIATION_FAILED.

15. PRIVACY AMPLIFICATION REQUIREMENTS

Use Toeplitz universal hashing.

Input:

reconciled key.

estimated leakage.

security margin.

Output length must be calculated by the configured privacy policy.

Output length must be positive.

If calculated output length ≤ 0 :

reject session.

Failure:

INSUFFICIENT_SECRET_ENTROPY.

The Toeplitz seed may be public.

The final secret key must remain in secure memory.

Store only its SHA-256 fingerprint.

16. TRUST ENGINE

The trust engine consumes ordered telemetry.

Telemetry must be sorted by event timestamp and sequence number.

For each channel maintain a rolling window.

Calculate:

current QBER.

EWMA.

positive CUSUM.

rolling variance.

QBER slope.

secret key rate delta when available.

telemetry freshness.

EWMA:

$EWMA_t = \alpha * qber_t + (1-\alpha) * EWMA_{(t-1)}$

CUSUM positive:

$S_t = \max(0, S_{(t-1)} + qber_t - baseline - drift)$

Trust states are deterministic.

UNKNOWN:

fewer than 5 valid telemetry events.

TRUSTED:

current QBER < static rejection threshold;

CUSUM below threshold;

telemetry fresh;

no integrity failures.

NOISY:

QBER above learned baseline;

CUSUM below threshold;

variance elevated;

no persistent positive drift.

DEGRADING:

positive QBER slope for configured consecutive windows;

or secret key rate falls by $\geq 30\%$.

SUSPICIOUS:

CUSUM threshold exceeded;

or combined evidence engine emits at least 2 attack indicators.

COMPROMISED:

QBER $\geq$ static rejection threshold;

or telemetry source cryptographic identity is revoked;

or physical QKD adapter reports compromised session.

The trust service must return evidence codes.

Required evidence codes:

INSUFFICIENT_HISTORY

QBER_STATIC_THRESHOLD_EXCEEDED

EWMA_BASELINE_DEVIATION

CUSUM_THRESHOLD_EXCEEDED

ROLLING_VARIANCE_ELEVATED

POSITIVE_QBER_TREND

SECRET_KEY_RATE_DROP

TELEMETRY_STALE

TELEMETRY_INTEGRITY_FAILURE

QKD_BACKEND_COMPROMISED

17. REAL ML-KEM REQUIREMENTS

Use ML-KEM-768.

The application must call the configured cryptographic provider.

Required operations:

keypair generation.

encapsulation.

decapsulation.

The service must verify:

decapsulated shared secret equals encapsulated shared secret during integration tests.

Never log:

secret key.

shared secret.

derived session key.

Store public key fingerprints only.

A random 32-byte value is not a valid replacement.

If provider unavailable:

PQC capability is false.

Hybrid mode is unavailable.

18. HYBRID KEY COMBINER

Inputs:

accepted QKD key handle.

ML-KEM shared secret handle.

session UUID.

channel UUID.

Alice UUID.

Bob UUID.

Construct input key material as:

QKD key bytes concatenated with ML-KEM shared secret bytes.

Use HKDF-SHA-256.

Salt:

32 random bytes generated by OS CSPRNG.

Info:

UTF-8 bytes of:

SHORLYNOT-QEDGE-HYBRID-V1 | session_id | channel_id | alice_id | bob_id

Output:

32 bytes.

The result is the AES-256-GCM session key.

If QKD key state is not ACCEPTED:

raise KEY_NOT_ACCEPTED.

If ML-KEM secret missing:

raise ML_KEM_SECRET_MISSING.

19. QKBNL POLICY ENGINE

The policy engine is the centrepiece.

Input:

channel state.

key state.

crypto mode.

telemetry validity.

node status.

lease request.

Decision table:

TRUSTED + ACCEPTED + HYBRID + valid telemetry: ALLOW lease.

TRUSTED + ACCEPTED + QKD_ONLY + valid telemetry: ALLOW lease.

NOISY + ACCEPTED + HYBRID: ALLOW lease with maximum TTL 60 seconds.

DEGRADING + ACCEPTED + HYBRID: ALLOW existing lease until expiry. Deny new QKD_ONLY lease. Request migration.

SUSPICIOUS: Deny new QKD lease. Request ML_KEM_ONLY migration. Revoke QKD_ONLY leases.

COMPROMISED: Reject QKD key. Revoke QKD-associated leases. Isolate channel. Allow ML_KEM_ONLY only if both nodes remain trusted.

UNKNOWN: Deny protected lease.

INVALID TELEMETRY: Deny state transition to TRUSTED. Preserve last safe policy for maximum 10 seconds. After 10 seconds migrate to ML_KEM_ONLY. If migration unavailable, revoke lease.

REVOKED NODE: Revoke all node leases. Isolate node.

No policy decision may be generated outside this table without a documented policy version change.

20. EDGEOS AGENT

The D-Link edge agent must not calculate BB84.

It must not run Qiskit.

It must not run PostgreSQL.

It must not run Redis.

It must not store raw QKD keys.

It receives signed enforcement commands.

Command schema:

version: uint8

request_id: 16-byte UUID

action: enum

lease_id: 16-byte UUID

source_ipv4: uint32

destination_ipv4: uint32

protocol: uint8

destination_port: uint16

expires_at: uint64 Unix seconds

crypto_mode: uint8

key_fingerprint: 32 bytes

signature_length: uint16

signature: variable bytes

Maximum command payload:

4096 bytes.

Agent validation order:

1. verify protocol version;
2. verify payload length;
3. verify signature;
4. verify request ID not previously executed;
5. verify expiry;
6. validate IP;
7. validate protocol;
8. validate port;
9. execute firewall operation;
10. inspect firewall state;
11. create receipt;
12. sign receipt;
13. return receipt.

A firewall command is successful only if post-execution inspection confirms the expected rule state.

For lease install:

create a dedicated QEDGE chain or firewall4-compatible set.

Rules must be identifiable by lease UUID.

For lease revoke:

remove the rule associated with lease UUID.

Never flush the entire firewall.

Never modify unrelated user rules.

21. ROUTER COMPATIBILITY GATE

Before any OpenWrt build:

verify exact:

manufacturer.

model.

hardware revision.

SoC.

flash size.

RAM.

OpenWrt target.

OpenWrt subtarget.

supported image format.

recovery mechanism.

Populate:

edgeos/compatibility/supported_devices.json

Schema:

```
{ "manufacturer": "D-Link", "model": "DSL-2750U", "hardware_revision": "EXACT_REVISION", "soc":  
"EXACT_SOC", "flash_mb": 0, "ram_mb": 0, "openwrt_target": "EXACT_TARGET", "openwrt_subtarget":  
"EXACT_SUBTARGET", "supported": false, "verified_source": "source reference description",  
"recovery_method": "description" }
```

Default supported value is false.

The build script must refuse to create a flash image when supported is false.

No automatic flashing script may be created.

Firmware installation remains a manual operator action.

22. FINANCIAL WORKLOADS

Payment service:

POST /payments

Input:

transaction_id UUID

sender string 1-100

receiver string 1-100

amount Decimal positive maximum 999999999999.99

currency exact 3 uppercase ASCII characters

timestamp UTC timestamp

The payload must traverse the QKBNL-controlled path.

Identity service:

POST /identity/verify

Input:

request_id UUID

subject_id string

challenge base64

Interbank service:

persistent secure message stream.

Archive service:

authenticated encrypted file transfer.

Maximum test file size:

2 GB.

The services must reject application traffic if the secure channel is unavailable.

They must not bypass EdgeOS.

23. CRYPTO ASSET DISCOVERY

Source scanner:

Supported extensions:

.py .java .js .ts .go .rs .c .cpp .cs .conf .yaml .yml .json

Detect explicit references to:

RSA

ECDSA

ECDH

DSA

Diffie-Hellman

TLS protocol versions

known cryptographic library configuration strings

Certificate scanner:

PEM.

DER.

CRT.

Extract:

subject.

issuer.

serial fingerprint.

public key algorithm.

key size.

signature algorithm.

valid from.

valid until.

TLS scanner:

Scan one explicitly provided host and port.

Collect negotiated protocol and certificate chain.

No subnet scanning.

Config scanner:

Nginx.

Apache.

OpenSSL configuration.

Java security configuration.

Every discovery result must contain:

scanner type.

target.

file or endpoint.

line number when applicable.

algorithm.

evidence snippet maximum 200 characters.

24. RISK ENGINE

Risk score range:

0 to 100.

Formula:

algorithm vulnerability weight: 35%.

data retention weight: 20%.

asset criticality weight: 20%.

external exposure weight: 15%.

migration complexity weight: 10%.

Each component normalized 0 to 100.

Final:

0-24.999: LOW

25-49.999: MEDIUM

50-74.999: HIGH

75-100: IMMEDIATE

Weights are constants.

The dashboard must show the exact component scores.

No opaque AI risk score.

25. DASHBOARD RULES

The dashboard is read-oriented.

It may trigger explicit operator actions through APIs.

It must not contain locally generated metric values.

Required pages:

Overview.

Quantum.

Channels.

Leases.

Crypto.

Edge.

Assets.

Audit.

Overview shows:

physical QKD capability.

real QPU capability.

ML-KEM capability.

EdgeOS status.

active channels.

active leases.

crypto mode counts.

critical audit events.

Quantum page shows:

backend type.

hardware provenance.

QKD sessions.

QBER.

key lifecycle.

No physical QKD badge when backend is BB84_PROTOCOL.

Channels page shows:

channel state.

QBER.

EWMA.

CUSUM.

evidence.

Leases page shows:

lease ID.

source.

destination.

protocol.

port.

key fingerprint.

crypto mode.

state.

expiry.

edge rule ID.

Crypto page shows:

ML-KEM capability.

active modes.

migration events.

Edge page shows:

device identity.

hardware revision.

firmware version.

agent version.

firewall backend.

last receipt.

Assets page shows:

discovered cryptography.

risk score.

migration phase.

Audit page shows append-only audit events.

26. ERROR HANDLING MATRIX

MODULE: DATABASE

Connection failure: startup fails. HTTP health 503. No application mutation accepted.

Transaction serialization conflict: retry maximum 3 times. Delays 50 ms, 100 ms, 200 ms. Then return 409 TRANSACTION_CONFLICT.

Unique violation: map to deterministic 409 code.

Foreign key violation: return 409 RESOURCE_REFERENCE_CONFLICT.

MODULE: REDIS

Startup unavailable: startup fails.

Runtime unavailable: telemetry endpoint returns 503. Do not process replay-sensitive telemetry without replay cache.

MODULE: BB84

Empty sifted key: reject session. BB84_EMPTY_SIFTED_KEY.

Empty QBER sample: reject session. QBER_SAMPLE_EMPTY.

Reconciliation failure: reject. RECONCILIATION_FAILED.

Privacy output ≤ 0 : reject. INSUFFICIENT_SECRET_ENTROPY.

Key confirmation mismatch: reject. KEY_CONFIRMATION_FAILED.

MODULE: PHYSICAL QKD

TLS failure: QKD_BACKEND_TLS_FAILURE.

Authentication failure: QKD_BACKEND_AUTH_FAILURE.

Unsupported protocol: QKD_PROTOCOL_UNSUPPORTED.

Timeout: QKD_BACKEND_TIMEOUT.

Malformed response: QKD_BACKEND_INVALID_RESPONSE.

Device reports compromised: session rejected. channel COMPROMISED.

MODULE: IBM QUANTUM

Missing credentials: capability unavailable.

Backend not found: QUANTUM_BACKEND_NOT_FOUND.

Simulator selected in hardware mode: QUANTUM_SIMULATOR_REJECTED.

Job failure: QUANTUM_JOB_FAILED.

Result hash failure: QUANTUM_RESULT_INTEGRITY_FAILURE.

MODULE: TELEMETRY

Invalid JSON: 422.

Invalid signature: 401.

Revoked node: 403.

Stale packet: 409 TELEMETRY_STALE.

Duplicate sequence: 409 DUPLICATE_SEQUENCE.

Replay nonce: 409 NONCE_REPLAY.

Sequence regression: 409 SEQUENCE_REGRESSION.

Unknown channel: 404.

MODULE: TRUST

Insufficient events: UNKNOWN.

Calculation NaN: TRUST_CALCULATION_INVALID. Channel becomes UNKNOWN. Audit HIGH.

Out-of-order event: reject before trust calculation.

MODULE: ML-KEM

Provider unavailable: 503.

Key generation failure: 502 ML_KEM_KEYGEN_FAILED.

Encapsulation failure: 502 ML_KEM_ENCAPSULATION_FAILED.

Decapsulation failure: 502 ML_KEM_DECAPSULATION_FAILED.

MODULE: HYBRID KDF

Rejected QKD key: 403 KEY_NOT_ACCEPTED.

Missing ML-KEM secret: 409 ML_KEM_SECRET_MISSING.

Output failure: 500 KEY_DERIVATION_FAILED.

MODULE: LEASE

Duplicate active lease: 409.

Expired key: 403.

Rejected key: 403.

Edge unavailable: 503.

Edge execution failure: lease FAILED. 502.

Invalid receipt: lease FAILED. 502 EDGE_RECEIPT_INVALID.

MODULE: EDGE

Invalid signature: reject command. No firewall mutation.

Duplicate request ID: return previous signed receipt.

Expired command: reject.

Firewall binary unavailable: return FAILED receipt.

Firewall rule installation failure: return FAILED.

Post-install verification failure: remove attempted rule. return FAILED.

Rule revoke target absent: return idempotent SUCCEEDED only if rule is confirmed absent.

MODULE: ASSET SCANNER

Path missing: 404.

Permission denied: 403.

Unsupported file: 422.

TLS connection failure: 502.

Malformed certificate: 422.

MODULE: MIGRATION

No assets: return empty plan.

Missing benchmark data: recommendation may be created. measured latency remains null. UI must show NOT MEASURED.

MODULE: AUDIT

Hash chain failure: stop mutating security operations. enter AUDIT_INTEGRITY_FAILURE mode. return 503 for lease and crypto mutations.

27. CONCURRENCY RULES

Telemetry sequence validation must use Redis atomic operations.

Key lifecycle transition must use PostgreSQL row locks.

Crypto rekey must lock crypto_sessions row.

Lease creation must enforce uniqueness for equivalent ACTIVE lease tuples.

Lease expiry worker must be idempotent.

Edge command request IDs must be idempotent.

Audit event creation must serialize hash-chain updates.

Two concurrent trust decisions for one channel must be ordered by telemetry sequence.

A lower telemetry sequence may never overwrite a newer channel state.

28. SECURITY REQUIREMENTS

All inter-node HTTP uses TLS.

Node-sensitive endpoints require mutual TLS.

Dashboard uses a separate operator authentication mechanism.

Development operator authentication:

username/password.

Password hashing:

Argon2id.

Session:

HttpOnly cookie.

Secure true in production.

SameSite Strict.

Session expiry:

15 minutes idle.

Maximum session lifetime:

8 hours.

CSRF token required for dashboard mutation requests.

Security headers:

Content-Security-Policy.

X-Content-Type-Options: nosniff.

Referrer-Policy: no-referrer.

Permissions-Policy.

No wildcard CORS.

Rate limits:

telemetry: 100 requests/second/node.

dashboard: 60 requests/minute/session.

mutation endpoints: 30 requests/minute/operator.

Rate-limit rejection: 429.

29. TEST REQUIREMENTS

Unit coverage minimum:

90% for:

trust.

policy.

lease.

key lifecycle.

telemetry validation.

hybrid KDF.

Minimum total backend coverage:

80%.

Required invariant tests:

rejected_key_never_creates_lease

expired_key_revokes_lease

revoked_session_revokes_all_leases

invalid_telemetry_never_trusts_channel

rejected_qkd_key_never_enters_hybrid_kdf

zeroized_key_never_retrievable

traffic_waits_for_key_confirmation

edge_failure_never_activates_lease

dashboard_state_matches_backend

physical_qkd_never_faked

Required attack tests:

telemetry replay.

telemetry tampering.

sequence regression.

nonce replay.

fake node.

expired key lease.

rejected key lease.

policy replay.

duplicate edge command.

invalid edge receipt.

downgrade request.

lockdown recovery attempt.

30. REQUIRED LIVE DEMONSTRATION

Initial state:

Alice cannot reach Bob payment service.

Command from Alice must fail.

Reason:

NO ACTIVE QKBNL.

Start secure session.

BB84 or physical QKD backend executes according to configured capability.

Key lifecycle completes.

If accepted:

QKBNL is requested.

EdgeOS installs real firewall permission.

Signed receipt is verified.

Lease becomes ACTIVE.

Alice reaches Bob.

Trigger suspicious quantum security state through the configured test mechanism.

If using physical QKD:

use actual hardware-reported channel state.

If no physical QKD hardware:

the demonstration must explicitly say BB84 PROTOCOL TEST MODE.

The control plane revokes the QKD-bound lease.

D-Link removes the actual firewall rule.

Alice-to-Bob protected traffic stops.

ML-KEM-768 establishes a real PQC secret.

Crypto session migrates to ML_KEM_ONLY.

A replacement QKBNL is issued.

D-Link installs the replacement policy.

Traffic resumes.

The demonstration must display:

real API events.

real lease IDs.

real edge receipts.

real firewall state.

real ML-KEM execution.

real packet interruption.

real recovery latency.

No video-only fake transition.

31. EXPERIMENT OUTPUTS

Every experiment writes CSV.

Required CSV columns include:

experiment_id.

run_id.

timestamp.

backend_type.

channel_id.

session_id.

qber.

ewma.

cusum.

variance.

channel_state.

key_state.

crypto_mode.

lease_state.

detection_time_ms.

migration_time_ms.

edge_execution_time_ms.

packet_loss_count.

application_interruption_ms.

cpu_percent.

memory_mb.

All unavailable values are empty.

Never insert fabricated zero values.

Required generated plots:

QBER over time.

EWMA over time.

CUSUM over time.

trust-state timeline.

lease-state timeline.

packet interruption timeline.

crypto migration latency distribution.

edge execution latency distribution.

ML-KEM encapsulation latency.

ML-KEM decapsulation latency.

asset risk distribution.

32. FILE-BY-FILE INTEGRITY GUIDE

THE FOLLOWING PROTOCOL IS MANDATORY FOR THE AGENTIC IDE.

Before creating a file:

1. inspect the repository tree;

2. determine whether the file already exists;
3. if the file exists, open the entire file;
4. identify all imports;
5. identify all exported names;
6. identify all callers;
7. identify all tests;
8. identify database model dependencies;
9. identify schema dependencies;
10. only then modify the file.

Before modifying a function:

1. search the complete repository for the function name;
2. list every caller;
3. inspect argument expectations;
4. inspect return-type expectations;
5. inspect exception handling;
6. inspect tests;
7. modify implementation;
8. update all incompatible callers in the same change;
9. run type checking;
10. run targeted tests.

For SQLAlchemy model changes:

1. inspect model;
2. inspect Pydantic schemas;
3. inspect repository;
4. inspect service;
5. inspect router;
6. inspect frontend type;
7. create Alembic migration;
8. run migration on empty database;
9. run migration on current development database;
10. run database tests.

For API changes:

1. inspect router;
2. inspect schema;
3. inspect service;
4. inspect repository;
5. inspect frontend API client;
6. inspect frontend types;
7. inspect integration tests;
8. modify all affected layers;
9. regenerate OpenAPI;
10. validate frontend types.

For enum changes:

search the entire repository.

Update:

database enum.

Pydantic schema.

policy engine.

frontend type.

tests.

migration.

Never update one enum definition in isolation.

For EdgeOS protocol changes:

update:

Python command serializer.

C protocol header.

C protocol parser.

receipt schema.

integration test vectors.

protocol documentation.

The Python and C protocol representations must be validated against identical binary test vectors.

33. ZERO-ERROR BACKEND FIXING PROTOCOL

Before compiling or deploying, the agent must open every backend file sequentially. The agent must execute a line-by-line static analysis, diagnose any hidden syntax/type mismatches, and apply immediate corrections. The backend must achieve a 100% error-free compilation state before credit depletion.

MANDATORY EXECUTION ORDER:

1. run `python -m compileall control_plane nodes workloads experiments tests;`

2. fix every syntax error;

3. run `ruff check .;`

4. fix every Ruff error;

5. run `mypy --strict control_plane nodes workloads;`

6. fix every mypy error;

7. run Alembic migration validation;

8. run `pytest tests/unit -q;`

9. fix every unit failure;

10. run `pytest tests/integration -q;`

11. fix every integration failure;

12. compile EdgeOS C code with:

```
-std=c17 -Wall -Wextra -Werror -Wpedantic ;
```

1. run clang-tidy;

2. run cppcheck;

3. run frontend TypeScript compilation;

4. run Vitest;

5. run Playwright;

6. run full `pytest -q;`

7. generate coverage report;

8. verify coverage requirements;

9. run Docker Compose health checks;

10. execute end-to-end lease lifecycle;
11. execute end-to-end ML-KEM migration;
12. execute telemetry replay attack;
13. execute edge receipt validation;
14. inspect logs for secret leakage;
15. scan repository for:

`pass`

`TODO`

`FIXME`

`mock`

`fake`

`demo_key`

`os.urandom(32)` used as PQC

`verify=False`

`except Exception: pass`

1. inspect every match manually;
2. remove prohibited placeholder implementations;
3. generate final build report.

The agent may not declare completion while any mandatory command fails.

34. INVARIABLE FILE RULES

Every Python file:

must contain complete imports.

must contain type annotations.

must not use wildcard imports.

must not create circular imports.

must not contain unused imports.

must not contain placeholder functions.

Every repository file:

performs database access only.

contains no policy logic.

Every service file:

contains business logic.

does not directly construct HTTP responses.

Every router file:

validates request schema.

calls services.

maps domain exceptions.

contains no SQL queries.

Every Pydantic schema:

uses explicit field types.

uses validation constraints.

must not use untyped dict where a stable schema exists.

Every C source file:

must compile with warnings treated as errors.

must validate input length before memory access.

must check every allocation.

must check every system-call return value.

must use bounded string operations.

Every frontend file:

must use generated or explicitly synchronized API types.

must not hardcode backend metrics.

must display loading state.

must display API error state.

must display unavailable capability state.

35. AGENT BUILD PHASES

PHASE 0

Repository bootstrap.

Configuration.

PostgreSQL.

Redis.

Logging.

Health endpoint.

Static analysis configuration.

PHASE 1

Database models.

Alembic.

Repositories.

Schemas.

PHASE 2

Node identity.

ML-DSA.

QTP validation.

Replay protection.

Audit hash chain.

PHASE 3

BB84 protocol.

QBER.

Reconciliation.

Privacy amplification.

Key confirmation.

PHASE 4

Trust engine.

EWMA.

CUSUM.

Variance.

Evidence engine.

PHASE 5

ML-KEM-768.

Secure key handles.

AES-GCM.

Hybrid KDF.

Crypto sessions.

PHASE 6

QKBNL policy engine.

Lease service.

Enforcement commands.

Receipts.

PHASE 7

EdgeOS C agent.

Firewall integration.

Lease installation.

Lease revocation.

Receipt signing.

PHASE 8

Alice node.

Bob node.

Secure channel.

Payment workload.

Identity workload.

Interbank workload.

Archive workload.

PHASE 9

Physical QKD adapter contract.

ETSI-oriented adapter boundary.

IBM Quantum adapter.

Capability reporting.

PHASE 10

Crypto asset discovery.

Risk engine.

Migration planner.

PHASE 11

Dashboard.

PHASE 12

Experiments.

Attack tests.

Formal model.

PHASE 13

Full integrity scan.

End-to-end validation.

Documentation.

The agent must complete and verify each phase before entering the next phase.

36. DEFINITION OF DONE

The project is complete only when:

1. All mandatory files exist.
2. No prohibited placeholder implementation exists.

3. PostgreSQL migrations pass.
4. Redis replay protection passes concurrency tests.
5. BB84 protocol completes successful sessions.
6. BB84 compromised sessions reject key material.
7. Reconciliation executes.
8. Privacy amplification executes.
9. Key confirmation executes.
10. ML-KEM-768 encapsulation and decapsulation execute through the real provider.
11. Hybrid KDF uses accepted QKD material and ML-KEM material.
12. Rejected QKD material cannot enter the hybrid KDF.
13. QTP rejects tampered telemetry.
14. QTP rejects replayed telemetry.
15. Trust engine produces deterministic evidence.
16. QKBNL cannot activate without valid policy state.
17. D-Link EdgeOS executes a real firewall mutation.
18. EdgeOS verifies installed firewall state.
19. EdgeOS signs an execution receipt.
20. Control plane verifies the receipt.
21. Network lease becomes ACTIVE only after receipt verification.
22. Lease revocation physically stops protected traffic.
23. ML-KEM migration creates a new cryptographic session key.
24. Replacement lease physically restores protected traffic.
25. Dashboard displays only backend-derived state.

26. Physical QKD is never claimed when no physical QKD backend exists.
27. Real QPU is never claimed when a simulator executes.
28. Crypto asset scanner produces evidence-backed findings.
29. Risk engine produces deterministic scores.
30. Migration planner uses stored asset and benchmark data.
31. Audit hash chain verifies.
32. Unit coverage requirements pass.
33. Integration tests pass.
34. Security tests pass.
35. Edge tests pass.
36. End-to-end tests pass.
37. Python compilation passes.
38. Ruff passes.
39. mypy strict passes.
40. C compilation passes with `-Werror`.
41. clang-tidy passes.
42. cppcheck passes.
43. TypeScript compilation passes.
44. Vitest passes.
45. Playwright passes.
46. Full test suite passes.
47. No raw cryptographic secret appears in logs.
48. No fake quantum provenance exists.

49. No fake PQC implementation exists.

50. The live QKBNL demonstration succeeds using actual packet forwarding through the physical D-Link edge node.

THE AGENT MUST NOT OUTPUT "PROJECT COMPLETE" UNTIL ALL 50 CONDITIONS ARE VERIFIED.

37. FINAL AGENT DIRECTIVE

Build the system phase-by-phase.

Do not optimize for visual appearance before backend correctness.

Do not skip failing tests.

Do not replace unavailable hardware with fabricated values.

Do not reinterpret physical QKD as gate-model quantum processor execution.

Do not remove security checks to make tests pass.

Do not comment out failing tests.

Do not weaken type checking.

Do not use `Any` to suppress mypy failures unless an external untyped library boundary requires it and the boundary is wrapped in a typed adapter.

Do not silently change architecture.

When a dependency is unavailable, expose the capability as unavailable.

When the D-Link hardware revision is unknown, build the EdgeOS agent and OpenWrt package but refuse firmware image generation.

When physical QKD hardware is unavailable, build and verify the adapter contract but report `PHYSICAL_QKD: UNAVAILABLE`.

The project is successful when quantum-security and cryptographic state produces verifiable, real packet-level consequences at the physical edge.

The central implementation chain is:

QKD OR BB84 SECURITY STATE -> AUTHENTICATED TELEMETRY -> TEMPORAL TRUST DECISION -> KEY
LIFECYCLE STATE -> CRYPTOGRAPHIC MODE -> QKBNL POLICY -> D-LINK EDGEOS -> REAL FIREWALL RULE ->
REAL PACKET FORWARDING STATE

This chain must be fully traceable through immutable audit events.

END OF MASTER PRD.